基于循环神经网络与 Transformer 的古诗生成实验报告

1. 实验目的

本实验基于中文古诗数据集，使用多种神经网络模型进行古诗自动生成任务研究。实验主要目标包括：

1. 理解循环神经网络（RNN）在自然语言生成中的基本原理；
2. 掌握 LSTM、GRU、Attention-LSTM 与 Transformer 等模型结构；
3. 比较不同网络结构、隐藏层数、隐藏层维度以及激活函数对生成效果的影响；
4. 观察训练过程中 Loss 的变化情况；
5. 分析不同模型生成古诗的语言流畅性与意境连贯性。

---

2. 实验环境

```
Python 3.13
PyTorch
CUDA
matplotlib
numpy

DEVICE = torch.device("cuda")

---
```

3. 数据集与预处理

3.1 数据来源

实验使用《全宋诗》数据集：

```
poet.song.40000.json
poet.song.41000.json
poet.song.42000.json
poet.song.43000.json
```

数据以 JSON 格式存储，每首诗包含 paragraphs 字段。

3.2 数据筛选

实验仅保留：

- 七言绝句
- 共四句
- 每句七个汉字

筛选逻辑：

```
if len(lines) == 4 and all(len(line) == 7 for line in lines):
```

最终筛选得到：

```
七言绝句数量: 901
字符表大小: 2824
```

3.3 数据表示

采用字符级建模（Character-level Modeling）：

```
char2idx = {ch: i for i, ch in enumerate(chars)}
idx2char = {i: ch for ch, i in char2idx.items()}
```

输入序列长度：

```
SEQ_LEN = 32
```

表示模型每次读取连续 32 个字符作为输入，并学习预测其后续字符。

训练目标：

给定前面的字符序列，预测下一个字符。例如输入为“明月松间照”，目标则为“月松间照清”，模型通过这种方式学习“给定前面的字符序列，预测下一个字符”的规律。经过大量古诗文本训练后，模型能够逐渐学习古诗中常见的字词搭配、句式结构和语言风格，因此在生成阶段只需要给定起始词“明月”，模型就可以逐字预测后续内容，最终生成一首完整的七言绝句。

---

4. 网络模型设计

本实验实现并比较了五种模型：

模型	特点
RNN	基础循环神经网络
LSTM	引入门控机制，解决长依赖问题
GRU	简化版 LSTM
Attention-LSTM	在 LSTM 基础上加入注意力机制
Transformer	基于 Self-Attention 的结构

---

5. 实验参数设置

5.1 基础参数

```
BATCH_SIZE = 64
EPOCHS = 100
```

```
LR = 0.001
EMBED_DIM = 128
SEQ_LEN = 32
DROPOUT = 0.3
TOP_K = 20
```

5.2 扫参设置

网络结构

```
MODEL_TYPES = [
    "LSTM",
    "GRU",
    "RNN",
    "AttentionLSTM",
    "Transformer"
]
```

隐藏层数

```
NUM_LAYERS_LIST = [1, 2, 3]
```

隐藏层维度

```
HIDDEN_DIM_LIST = [128, 256]
```

激活函数

```
ACTIVATION_LIST = ["Tanh", "ReLU", "GELU"]
```

总实验数量：

```
84 组实验
```

---

6. 实验结果分析

6.1 最优实验结果

实验中记录了每组模型每一轮训练的 Loss，并将结果保存为 `loss_history.txt`、`loss_history.csv` 和 `loss_curve.png`。根据 `summary.csv` 中的统计结果，Best Loss 最低的前五组模型如下：

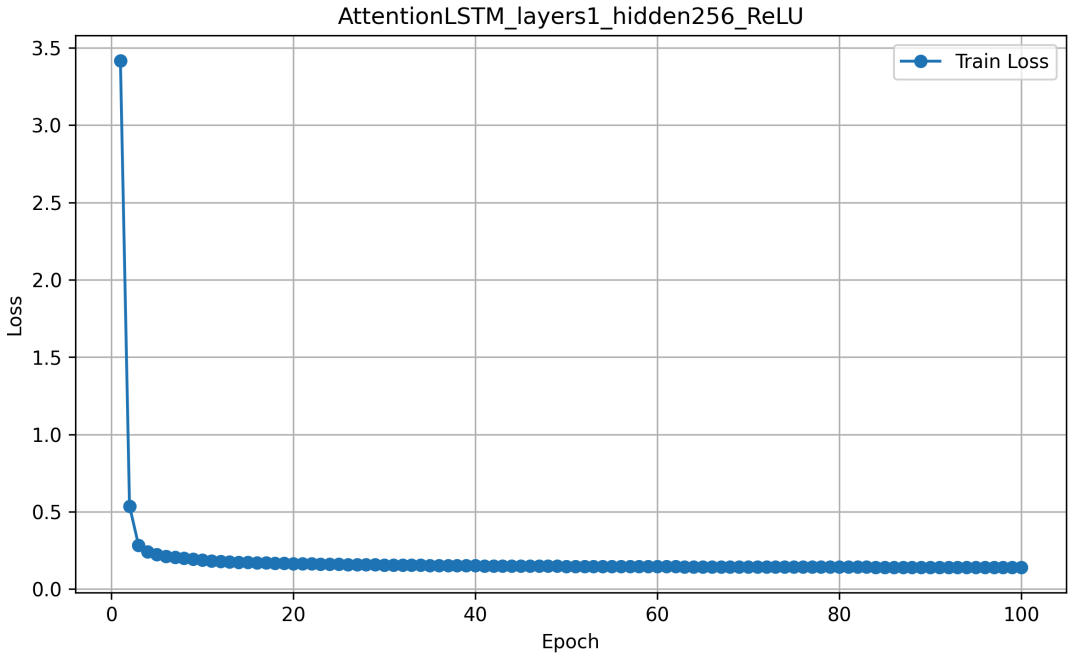
排名	模型	Best Loss
1	AttentionLSTM_layers1_hidden256_ReLU	0.140390
2	LSTM_layers1_hidden256_GELU	0.140929
3	AttentionLSTM_layers1_hidden256_GELU	0.141390
4	LSTM_layers1_hidden256_Tanh	0.142093
5	LSTM_layers1_hidden256_ReLU	0.142276

可以发现：

1. Attention-LSTM 效果最好；
2. hidden_dim=256 明显优于 128；
3. ReLU 与 GELU 效果优于 Tanh；
4. 单层网络反而效果较好。

6.2 最优loss曲线分析

下面展示 Best Loss 最低模型 `AttentionLSTM_layers1_hidden256_ReLU` 的 Loss 曲线。



该模型的 Loss 在训练初期下降较快，说明模型能够迅速学习到字符级语言建模的基本规律；随后 Loss 下降速度逐渐变慢，并在后期趋于稳定，表明模型已经基本收敛。

从整体趋势来看，Loss 曲线没有明显震荡，说明训练过程较为稳定。最终 Best Loss 达到 0.140390，说明模型已经较好地学习了古诗文本中的字符分布、句式结构和语言风格。

7. 不同模型对比分析

7.1 RNN

优点：

- 结构简单；
- 参数量较小；
- 训练速度快。

缺点：

- 长距离依赖能力较差；
- 容易生成重复文本；

- 意境连续性不足。

7.2 LSTM

LSTM 通过输入门、遗忘门与输出门增强记忆能力。

实验结果表明：

- Loss 明显低于 RNN；
- 生成诗句更加流畅；
- 上下句语义关联增强。

7.3 GRU

GRU 相比 LSTM：

- 参数更少；
- 收敛更快；
- 训练速度较高。

但实验中：

- 最终 Loss 略高于 LSTM；
- 意境连贯性稍弱。

说明在古诗生成任务中，LSTM 的记忆能力更适合长文本生成。

7.4 Attention-LSTM

Attention-LSTM 在 LSTM 基础上增加 Multi-Head Attention：

```
self.attention = nn.MultiheadAttention(...)
```

作用：

- 强化上下文建模；
- 提高长距离依赖能力；
- 增强句间联系。

实验结果中：

- 获得最低 Loss；
- 诗句结构更自然；
- 上下句联系更强。

说明注意力机制能够有效提升古诗生成质量。

7.5 Transformer

Transformer 使用 Self-Attention 进行全局建模：

```
nn.TransformerEncoder
```

优点：

- 全局上下文能力强；
- 并行计算效率高；
- 理论表达能力强。

但实验中：

- Loss 未明显优于 Attention-LSTM；
- 小数据集下容易不稳定；
- 参数量较大。

原因可能是：

- 数据集仅 901 首诗；
- Transformer 更依赖大规模数据。

---

8. 隐藏层维度分析（hidden_dim）

实验比较了：

```
hidden_dim = 128
hidden_dim = 256
```

结果发现：

- hidden_dim=256 效果明显更好；
- 更大的隐藏维度能够学习更丰富的语义信息；
- 生成结果更加稳定。

但同时：

- 显存占用更大；
- 训练时间更长；
- 过拟合风险增加。

---

9. 网络层数分析（num_layers）

实验测试：

```
num_layers = 1, 2, 3
```

结果表明：

- 1 层模型效果最好；
- 层数增加后提升有限；

原因可能是：

- 数据集较小；
- 古诗文本长度较短；
- 网络层数增加时，100轮内未收敛到最优

---

10. 激活函数分析

实验比较：

```
Tanh
ReLU
GELU
```

结果发现：

- GELU 与 ReLU 效果最好；
- Tanh 收敛较慢；
- GELU 在 Transformer 中表现较稳定。

原因：

- GELU 在深层网络中梯度更平滑；
- ReLU 收敛速度较快；
- Tanh 存在梯度饱和问题。

---

11. 古诗生成结果分析

训练过程中，每 10 个 epoch 自动生成古诗，其中最优模型生成结果如下：

```
Epoch 10:
明月滿庭無隱伏，夜深惟有此君知。
黃紙恩書下紫宸，豈惟羣罪許從新。

Epoch 20:
明月滿庭無隱伏，夜深惟有此君知。
黃紙恩書下紫宸，豈惟羣罪許從新。

Epoch 30:
明月似瀟湘湖外，寄吾家路語荷尋。
仙山青春春自饒，未容取次老僧敲。

Epoch 40:
明月爲高燭一道，清光照萬山水澄。
山色映青天靜坐，茅堂未欲眠好玩。

Epoch 50:
明月似瀟湘湖外，青山案几横卷簾。
幽坐少逢迎一鈎，香餌看魚上萬頃。

Epoch 60:
明月臨汀氣候譌，桃花零落發林阿。
也知欲趁春風媚，争奈窮州地暖何。
```

```
Epoch 70:
明月讀離騷空谷，秋寒同子午白月。
紛紛洗清露遙想，燈花結夜愁鬱金。

Epoch 80:
明月滿庭無隱伏，夜深惟有此君知。
黄紙恩書下紫宸，豈惟羣罪許從新。

Epoch 90:
明月夜竿頭絲動，得魚時草心回暖。
春風香柳眼半開，春日長老禪隱几。

Epoch 100:
明月應無閑事掛，心頭老來無事掩。
柴扉林下閑閑何，所爲莫道茅庵無。
```

可以观察到：

- 早期生成结果随机性较强；
- 后期逐渐形成古诗结构；
- 模型开始学习意境与对仗；
- 部分诗句已经具有一定文学风格。

但仍存在：

- 语义不完全连贯；
- 个别字词组合不自然；
- 某些诗句重复。

---

12. 实验总结

本实验完成了基于多种神经网络结构的古诗自动生成任务，并进行了系统的参数对比实验。

实验结果表明：

1. LSTM 与 Attention-LSTM 明显优于基础 RNN；
2. Attention 机制能够提升上下文关联能力；
3. hidden_dim=256 的效果优于 128；
4. 小数据集下深层 Transformer 优势不明显；
5. 模型能够学习古诗的语言风格与句式结构。

最终最优模型为：

```
AttentionLSTM_layers1_hidden256_ReLU
```

其 Best Loss 为：

```
0.140390
```